

RETROSPECTIVE & STUDY GUIDE

Deploying SmartProperty to Azure

What was built, what broke, and how it maps to the AZ-400 DevOps Engineer Expert certification

Live site	https://smartproperties.tech
API	ca-smartproperty-api2...italynorth.azurecontainerapps.io
Stack	NestJS React + Vite FastAPI MongoDB
Commits	22, on branch chore/dead-code-cleanup
Net change	65 files, +5,921 / -45,706 lines
Monthly cost	approx. USD 5 against a USD 100 student credit

How to read this. Sections 1 to 5 are the engineering record - what was changed and why. Section 6 maps that work onto the five AZ-400 exam domains and, more usefully, lists what this project did *not* exercise. Section 7 turns those gaps into a study plan. The failures in section 5 are the most transferable part: almost none of them were code defects.

1. What the project started as

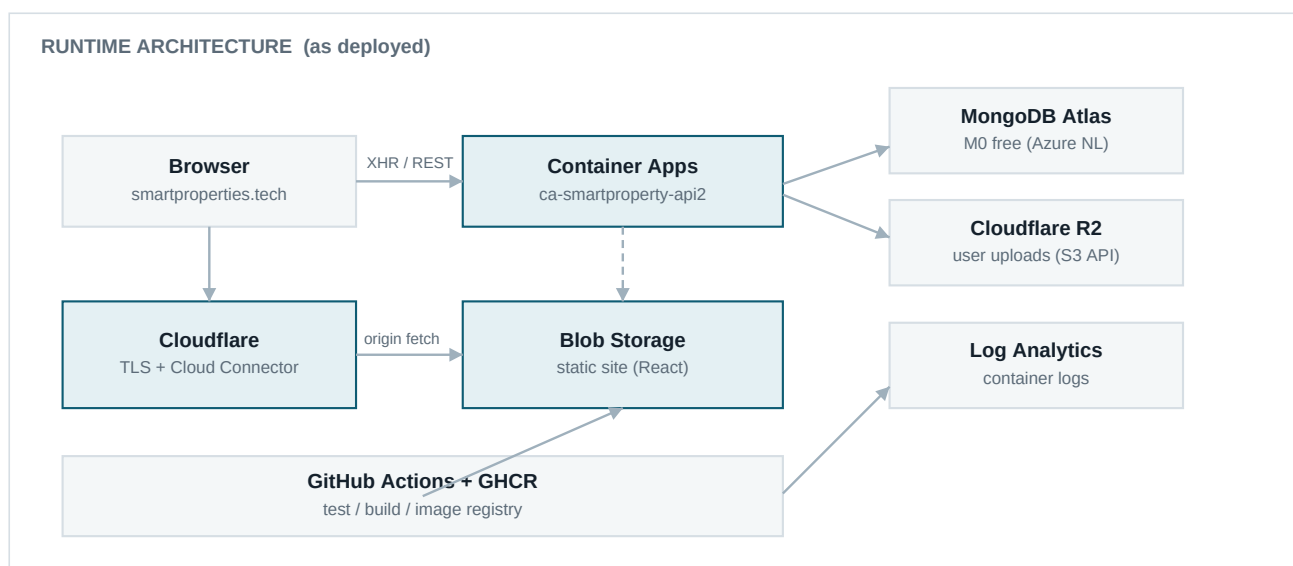
SmartProperty is a property-management platform: a NestJS REST API, a React single-page frontend built with Vite, a FastAPI service for AI features, and MongoDB for storage. Everything ran locally through Docker Compose. Nothing was deployed, and the work began as a security review rather than a deployment exercise.

Three findings reframed it:

Finding	Detail
Rate limiting was inert	ThrottlerModule was configured and five auth routes carried @Throttle decorators, but no APP_GUARD registered ThrottlerGuard. In NestJS the decorators do nothing without it, so login accepted unlimited attempts. The protection existed in code review and not at runtime.
Secrets were in public history	Google OAuth, Facebook app secret, reCAPTCHA secret and SendGrid keys were committed and readable across 18 commits in a public repository.
The frontend had never built	npm run build failed on 12 TypeScript errors, which masked a second failure: minify was set to esbuild, a package never present in any lockfile.

The transferable lesson. All three were invisible from reading the code. Configuration that looks present but is never wired up, and a build that no one has run, both pass review. Verification has to execute, not inspect.

2. Architecture as deployed



Every component below was chosen under a constraint rather than freely, which is the part worth understanding.

Component	Choice	Why this and not the obvious option
API	Azure Container Apps	Runs the container without operating Kubernetes. Consumption billing with an idle rate.
Frontend	Blob Storage static website	Azure Static Web Apps exists in five regions worldwide and the subscription policy permits none of them.
Database	MongoDB Atlas M0	Cosmos DB's RU API needs an index for every sorted field; the backend has 42 sorted queries and one endpoint that sorts on user input. The vCore tier has no such limit but its free tier is unavailable in any permitted region.

Component	Choice	Why this and not the obvious option
Object storage	Cloudflare R2	Speaks the S3 API, so the existing MinIO client was reused rather than rewritten. Free to 10 GB.
TLS / DNS	Cloudflare	Blob static websites cannot serve a certificate for a custom domain, and Azure's answer - Front Door - costs about USD 35/month.
Registry	GitHub Container Registry	Free. Azure Container Registry Basic is USD 5/month for the same job at this scale.
Queue / cache	Removed entirely	BullModule was configured but no queue was ever registered. Deploying Redis would have cost about USD 16/month to serve nothing.

The single largest saving came from deleting infrastructure, not provisioning it. Redis was in the compose file, so the obvious move was to deploy it. Reading the code showed no queue was ever registered. That is judgement, and it is what distinguishes running commands from understanding a system.

3. The work, in five phases

Phase 1 - Security hardening

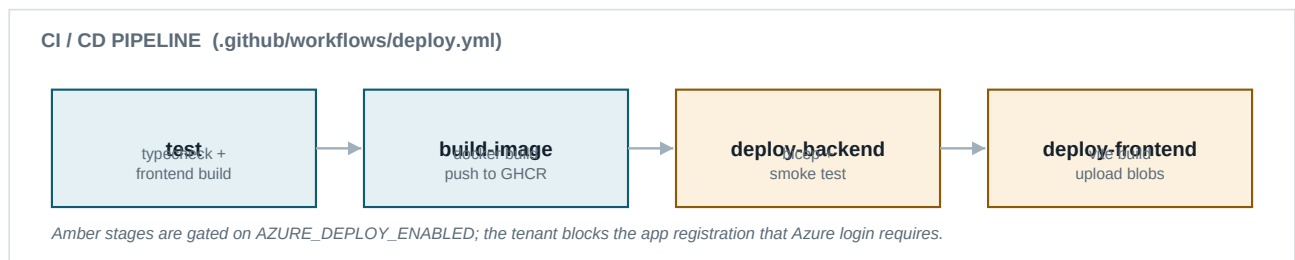
Registered ThrottlerGuard globally so the existing limits took effect. Removed hard-coded JWT fallback secrets from five files - the most serious in auth.module.ts, which was the *signing* key, meaning anyone who had read the repository could mint valid tokens. Disabled debug logging in production, closed an origin-less CORS hole, and made the seed password refuse to run outside development without SEED_PASSWORD.

Phase 2 - Removing what was not used

Deleted 35 dependencies with zero imports: the entire GraphQL stack (no GraphQLModule was ever registered), Bull and Redis, a redundant AWS S3 client, and 11 Radix packages whose UI components turned out to be hand-written. Dependency advisories fell from 54 to 27 on the frontend and 95 to 86 on the backend purely by deletion. Also fixed the 12 TypeScript errors and the minifier setting, so the production build succeeded for the first time.

Phase 3 - Infrastructure as code

Wrote infra/main.bicep declaring Log Analytics, a Container Apps environment, the API, the database and the storage account, plus a four-stage GitHub Actions workflow. The template creates the database and passes its own connection string to the container using listConnectionStrings() and listKeys(), which ARM resolves at deployment time - so no database credential is ever typed by hand or stored in CI. Required secrets dropped from ten to six.



Phase 4 - Deployment

Resource group, then az deployment group what-if to validate server-side without creating anything, then create. Five resources in 2m26s. The application then crash-looped for about an hour for reasons covered in section 5.

Phase 5 - Domain and storage

Moved DNS to Cloudflare, which terminates TLS and rewrites the Host header toward the storage endpoint. Connected Cloudflare R2 for uploads by adding two settings to the existing S3 client. Finally corrected seeded image URLs that pointed at localhost.

4. Cost model

Component	Service	Monthly (USD)
Frontend	Blob Storage static website + Cloudflare	~0.05
API	Container Apps, 1 replica warm	~3 - 5
Database	MongoDB Atlas M0	0.00

Component	Service	Monthly (USD)
Object storage	Cloudflare R2 (under 10 GB)	0.00
Logs	Log Analytics (5 GB/month free)	0.00
Registry	GitHub Container Registry	0.00
Total		~5

The binding constraint is time, not money: Azure for Students credit expires twelve months after activation regardless of the balance. At this rate the credit outlives its own expiry, so running the API warm rather than scaling to zero is the correct trade - a reviewer waiting twenty seconds for a cold start is a worse outcome than spending five dollars.

5. Every failure, and what caused it

Twelve distinct failures occurred. Two were application defects. The rest were platform constraints, propagation delays, or tooling that reported success while measuring the wrong thing. This distribution is itself the lesson.

#	Symptom	Actual cause	Class
1	Deployment validation failed with a policy violation	Subscription restricts resources to five regions; neither Static Web Apps nor the Cosmos vCore free tier exists in any of them	Governance
2	az ad app create - insufficient privileges	Tenant blocks app registration for student accounts, so no service principal and therefore no automated Azure deployment at all	Governance
3	Cannot make container package public	Organization disables public package visibility; registry credentials had to be passed to Container Apps instead	Governance
4	Image build failed at npm install	package.json has a postinstall hook, and the Dockerfile copied only package.json - not the scripts directory it runs	Defect
5	Container crash-looped: CORS_ORIGIN empty	Bicep passed an empty string; the Joi schema rejects it. Fixed by deriving the value from the storage account created in the same template	Config
6	az containerapp update rejected	Express environments do not support managed identity, revision suffixes, revision restarts or custom domains	Platform
7	Redeploy: 'resource already exists'	Deleting a container app leaves an internal artifact pending for over 15 minutes; worked around with a name suffix parameter	Platform
8	Secret updated but app kept old value	Express environments do not restart the container on a secret change. Only scaling min-replicas to 0 and back works	Platform
9	MongoServerSelectionError: SSL alert 80	Atlas rejects unlisted client IPs during the TLS handshake, not at authentication, so it presents as a certificate fault	Platform
10	Seeding failed: retryable writes unsupported	Cosmos rejects them; TypeORM discards retrywrites=false from the connection string when building driver options	Platform
11	Upload returned a URL that 404s	uploadFile built the public URL inline instead of calling getPublicUrl, so the R2 path rule never applied to uploads	Defect
12	Every listing image broken	The seed fell back to localhost:5173 for image URLs when run against production	Config

Three times my own tooling lied. Piping docker build through tail reported tail's exit code, so a failed build looked successful. A curl loop checking only the exit status reported an API as healthy while it returned 404. A silent str.replace left a workflow half-migrated. In each case the command succeeded - it just measured the wrong thing. Verify the assertion you actually care about: an HTTP status code, not a process exit code.

6. Mapping to AZ-400

AZ-400 (Designing and Implementing Microsoft DevOps Solutions) has five skill areas. Below is an honest assessment: what this project exercised, and what it did not. The second column matters more than the first, because gaps are where study time belongs.

Domain 1 - Configure processes and communications (10-15%)

Covered here	Not covered - study these
Documenting decisions and their rationale in commit messages and a maintained README; capturing operational knowledge such as the replica-cycling workaround so it is not rediscovered.	Azure Boards work items, backlogs and sprints; dashboards and team analytics; wikis; notification and service-hook integrations; defining and measuring DORA metrics (deployment frequency, lead time, change failure rate, MTTR).

Domain 2 - Design and implement source control (15-20%)

Covered here	Not covered - study these
Feature-branch workflow; small focused commits with explanatory messages; responding to automated code review across five rounds; .gitignore hygiene and removing tracked secrets from the index.	Azure Repos specifically; branch policies and required reviewers; purging secrets from history with git filter-repo (still outstanding here); Git LFS; monorepo versus multi-repo strategy; fork-based workflows and release branching models.

Domain 3 - Design and implement build and release pipelines (50-55%)

This is over half the exam. It is also where this project is thinnest, because the tenant blocked the credential that automated deployment requires.

Covered here	Not covered - study these
Multi-stage pipeline with job dependencies and outputs; multi-stage Dockerfile with a runtime-only dependency stage and a non-root user; container registry authentication; deploying by image digest rather than a mutable tag; a smoke test that fails the pipeline and dumps logs; infrastructure as code in Bicep, validated with what-if before applying.	Azure Pipelines YAML - the exam's primary vehicle, not GitHub Actions; self-hosted agents and agent pools; Azure Artifacts feeds and package versioning; deployment strategies (blue-green, canary, rolling); release gates and approvals ; environments and deployment jobs; feature flags; database deployment and migration strategy; testing in the pipeline - unit, integration and load.

Domain 4 - Develop a security and compliance plan (10-15%)

Covered here	Not covered - study these
OIDC federated credentials designed in - short-lived tokens rather than a stored secret - with the role assignment scoped to one resource group; secrets injected as container secrets and never written into the template; dependency scanning via npm audit, with 35 unused packages removed; least-privilege API tokens scoped to a single bucket; secure-by-default configuration that refuses to start when misconfigured.	Azure Key Vault and pipeline integration; managed identities (blocked by the express environment here); Microsoft Defender for Cloud; Azure Policy authoring - this project was on the receiving end of one, not the author; SAST/DAST tooling; container image scanning; compliance frameworks and audit reporting; credential rotation - identified here, still outstanding.

Domain 5 - Implement an instrumentation strategy (5-10%)

Covered here	Not covered - study these
Log Analytics workspace wired to the Container Apps environment; querying container logs with KQL to diagnose crash loops; health probes - readiness and liveness - driving container lifecycle; a build-time budget cap on the database to prevent silent overrun.	Application Insights - instrumentation, distributed tracing, dependency tracking; custom metrics and alert rules; availability tests; KQL beyond basic filtering; dashboards and workbooks; log-based alerting and action groups; cost analysis and budget alerts (recommended here but never configured).

7. Study plan

Ordered by exam weight against current gaps. The first item is by far the largest.

Priorit y	Topic	Why, and a concrete exercise
1	Azure Pipelines YAML	Over half the exam and entirely unexercised here, since this project used GitHub Actions. Rebuild this same deployment as azure-pipelines.yml with stages, environments and an approval gate before the deploy stage.
2	Deployment strategies	Blue-green, canary and rolling deployments. Container Apps supports traffic splitting across revisions natively - split 90/10 between two revisions and shift the weight.
3	Azure Key Vault	Container App secrets were used here; Key Vault with a managed identity is the pattern the exam expects. Move JWT secrets into Key Vault and reference them.
4	Application Insights	Instrumentation is a whole domain and only Log Analytics was touched. Add the Node SDK, generate traffic, then read a distributed trace end to end.
5	Azure Artifacts	Package feeds and versioning were never exercised. Publish the backend as a private npm package and consume it from a pipeline.
6	Azure Boards	Work item tracking and its links to commits and pull requests. Create a board and link these 22 commits to work items.
7	Azure Policy authoring	This project was <i>subject</i> to an allowed-regions policy. Write one - restrict allowed SKUs in a resource group and watch a deployment fail against it.
8	git filter-repo	Still genuinely outstanding on this repository. Practise purging a secret from history in a clone before doing it for real.

Outstanding on this project

Item	Status
Rotate the four credentials leaked in git history	Outstanding - Google OAuth, Facebook, reCAPTCHA, SendGrid remain readable in public history
Rotate the credentials shared during this work	Outstanding - the Atlas password still authenticated when last checked
Purge secrets from git history (git filter-repo)	Outstanding - must follow rotation, never replace it; requires a coordinated force-push
Automated deployment	Blocked - requires an Entra app registration the tenant does not permit for student accounts
OAuth callback URLs on the live host	Outstanding - social login will not work until updated
Dependency advisories	27 frontend and 86 backend remain; handlebars, react-router and multer are the ones that matter given what the code does
Deep links return HTTP 404	The SPA renders correctly, but crawlers see the wrong status code

The most useful thing to be able to explain. Not that it deployed, but that Redis was removed after reading the code rather than deployed because a compose file mentioned it; that five platform constraints were diagnosed from error messages that named none of them; and that the deployment pipeline holds no long-lived credential. Anyone can follow a tutorial to a green checkmark. Diagnosing an SSL alert that turns out to be an IP allowlist is the part that transfers.